CS302: Design and Analysis of Algorithms

Fall 2026 Project Master Deliverable

Complete Theoretical Solutions, Algorithm Implementations, Benchmarking, and System Design

Algorithm Research & Development Group (Group of 3)

November 8, 2026

Contents

1 Introduction and Project Overview **2**

2 Part 1: Theoretical Solutions [40 Marks] **3**

2.1 Problem 1(a): Min/Max Divide-and-Conquer Algorithm . . . 3
2.1.1 1. Algorithm Design and Pseudocode . . . 3
2.1.2 2. Recurrence Relation and Exact Closed-Form Solution . . . 3
2.1.3 3. Comparison with Brute-Force Algorithm . . . 4
2.2 Problem 1(b): Exponentiation a^n via Divide-and-Conquer . . . 4
2.2.1 1. Algorithm Design and Pseudocode . . . 4
2.2.2 2. Recurrence Relation and Multiplications Count . . . 5
2.2.3 3. Comparison with Brute-Force Algorithm . . . 5
2.3 Problem 1(c): Inversion Counting in $O(n \log n)$ . . . 5
2.3.1 1. Problem Statement . . . 5
2.3.2 2. Divide-and-Conquer Algorithm (Merge-and-Count) . . . 6
2.3.3 3. Time Complexity Proof . . . 6
2.4 Problem 1(d): Quicksort Best-Case and Worst-Case Analysis . . . 6
2.4.1 1. Best-Case Analysis . . . 7
2.4.2 2. Worst-Case Analysis . . . 7
2.5 Problem 1(e): 1D Closest-Pair Problem . . . 7
2.5.1 1. Direct Divide-and-Conquer Algorithm . . . 7
2.5.2 2. Critical Evaluation: Is it a Good Algorithm? . . . 8
2.6 Problem 1(f): Maximum Stock Profit via Divide-and-Conquer . . . 8
2.6.1 1. Problem Formulation . . . 8
2.6.2 2. Divide-and-Conquer Algorithm . . . 8
2.6.3 3. Recurrence and Time Complexity . . . 9
2.7 Problem 1(g): General Divide-and-Conquer Recurrence Analysis . . . 9
2.7.1 1. Recurrence Specification . . . 9
2.7.2 2. Derivation via Recursion Tree Analysis . . . 9
2.8 Problem 1(h): Selected Kleinberg & Tardos Exercises (Exercises 1, 2, and 3) . . . 10
2.8.1 Exercise 1: Median of Two Sorted Databases in $O(\log n)$ Queries . . . 10
2.8.2 Exercise 2: Counting Significant Inversions in $O(n \log n)$ . . . 11
2.8.3 Exercise 3: Fraudulent Bank Card Equivalence Testing in $O(n \log n)$ . . . 11

3 Part 2: Algorithm Implementation & Dataset Engineering [10 Marks] 12
3.1 Algorithm 1: Closest Pair of Points in 2D ($O(n \log n)$) 12
3.2 Algorithm 2: Karatsuba Integer Multiplication ($O(n^{\log_2 3})$) 12
3.3 Dataset Engineering 12

4 Part 3: Experimental Benchmarking & Results [5 Marks] 13
4.1 Closest Pair Benchmark Results 13
4.2 Integer Multiplication Benchmark Results 13

5 Part 4: User Interface Design & Visual Workbench [30 Marks] 14
5.1 UI Architecture and Design Principles 14

6 Part 5: Formal Project Report [15 Marks] 15

1 Introduction and Project Overview

This document presents the complete technical deliverable for the CS302 (Design and Analysis of Algorithms) Course Project. The project investigates the design, rigorous recurrence analysis, empirical benchmarking, and interactive visual prototyping of core **Divide-and-Conquer** algorithmic paradigms. The report encompasses all required parts:

1. **Part 1: Theoretical Analysis & Problem Solving [40 Marks]**: Exhaustive mathematical formulations, pseudocode, recurrence derivations, and asymptotic proofs for problems from Levitin (3rd Ed) and Kleinberg & Tardos (1st Ed).

2. **Part 2: Algorithm Implementation & Dataset Engineering [10 Marks]**: Comprehensive Python implementations of 2D Closest Pair ($O(n \log n)$) and Karatsuba Integer Multiplication ($O(n^{\log_2 3})$), with 20 distinct input datasets ($n > 100$).

3. **Part 3: Empirical Benchmarking & Application Testing [5 Marks]**: Detailed runtime evaluation, operation count metrics, and comparative speedup charts.

4. **Part 4: Graphical User Interface & Visual Workbench [30 Marks]**: Full architectural specification, state machine diagrams, and visual design of the interactive divide-and-conquer workbench.

5. **Part 5: Formal Conference-Style Project Report [15 Marks]**: A structured research paper following the standard IEEE/ACM format (Abstract, Introduction, Proposed System, Setup, Results, Conclusion, References).

2 Part 1: Theoretical Solutions [40 Marks]

2.1 Problem 1(a): Min/Max Divide-and-Conquer Algorithm

Reference: Levitin 3rd Ed, Ex 5.1(2), p.174

2.1.1 1. Algorithm Design and Pseudocode

The objective is to find both the maximum and minimum values in an array $A[0..n-1]$ using a divide-and-conquer strategy that minimizes key comparisons.

Listing 1: Divide-and-Conquer Min/Max Selection

```
Algorithm MinMaxDC(A, l, r):
    Input: Array A[l..r] of comparable keys, indices l <= r
    Output: Pair (min_val, max_val)

    if l == r:
        return (A[l], A[l])            # Base case 1: 1 element, 0
           comparisons
    elif l == r - 1:
        if A[l] < A[r]:                # Base case 2: 2 elements, 1
           comparison
            return (A[l], A[r])
        else:
            return (A[r], A[l])
    else:
        m = floor((l + r) / 2)
        (min1, max1) = MinMaxDC(A, l, m)
        (min2, max2) = MinMaxDC(A, m + 1, r)

        min_val = min(min1, min2)      # 1 element comparison
        max_val = max(max1, max2)      # 1 element comparison
        return (min_val, max_val)
```

2.1.2 2. Recurrence Relation and Exact Closed-Form Solution

Let $C(n)$ denote the number of element comparisons made on an input of size n. For $n = 2^k$ (where $k \geq 1$), the algorithm divides the array into two halves of size $n/2$, recursively finds the min and max of each half, and combines the results using exactly 2 comparisons (one to compare the two minima, and one to compare the two maxima).

The base cases are:

- $C(1) = 0$
- $C(2) = 1$

For $n > 2$ where $n = 2^k$:

$$C(n) = 2C\left(\frac{n}{2}\right) + 2 \tag{1}$$

Solving via Backward Substitution:

$$\begin{aligned} C(n) &= 2C(n/2) + 2 \\ &= 2[2C(n/4) + 2] + 2 = 4C(n/4) + 4 + 2 \\ &= 4[2C(n/8) + 2] + 4 + 2 = 8C(n/8) + 8 + 4 + 2 \\ &\vdots \\ &= 2^{k-1}C\left(\frac{n}{2^{k-1}}\right) + \sum_{i=1}^{k-1} 2^i \end{aligned}$$

Since $n = 2^k$, we have $\frac{n}{2^{k-1}} = 2$. Using the base case $C(2) = 1$:

$$\sum_{i=1}^{k-1} 2^i = 2 \cdot \frac{2^{k-1} - 1}{2 - 1} = 2^k - 2 = n - 2 \tag{2}$$

Substituting this back:

$$C(n) = 2^{k-1} \cdot (1) + (n - 2) = \frac{n}{2} + n - 2 = \frac{3}{2}n - 2 \tag{3}$$

2.1.3 3. Comparison with Brute-Force Algorithm

- **Standard Brute Force:** Scans sequentially, comparing each subsequent element against both the current minimum and current maximum:

$$C_{\text{brute}}(n) = 2(n - 1) = 2n - 2 \tag{4}$$

- **Divide-and-Conquer:** Achieves exactly:

$$C_{\text{D\&C}}(n) = \frac{3}{2}n - 2 \tag{5}$$

- **Efficiency Comparison:** The divide-and-conquer approach saves 25% of key comparisons, matching the information-theoretic lower bound for simultaneous min-max selection.

2.2 Problem 1(b): Exponentiation a^n via Divide-and-Conquer

Reference: Levitin 3rd Ed, Ex 5.1(3), p.174

2.2.1 1. Algorithm Design and Pseudocode

Binary exponentiation calculates a^n by exploiting:

$$a^n = \begin{cases} (a^{n/2})^2 & \text{if } n \text{ is even} \\ a \cdot (a^{\lfloor n/2 \rfloor})^2 & \text{if } n \text{ is odd} \end{cases} \tag{6}$$

Listing 2: Binary Exponentiation Algorithm

```
Algorithm PowerDC(a, n):
    Input: Base a in Reals, exponent n in Non-negative Integers
    Output: Value a^n

    if n == 0:
```

```
        return 1
    elif n == 1:
        return a
    else:
        half = PowerDC(a, floor(n / 2))
        if n % 2 == 0:
            return half * half            # 1 multiplication
        else:
            return a * half * half        # 2 multiplications
```

2.2.2 2. Recurrence Relation and Multiplications Count

Let $M(n)$ be the number of scalar multiplications required to compute a^n:

$$M(n) = M(\lfloor n/2 \rfloor) + \delta(n), \quad \text{where } \delta(n) = \begin{cases} 1 & \text{if } n \text{ is even} \\ 2 & \text{if } n \text{ is odd} \end{cases} \tag{7}$$

with base case $M(1) = 0$ (or $M(0) = 0$).

Closed-Form and Asymptotic Bound: Let $b(n)$ be the binary representation of n, with length $\lfloor \log_2 n \rfloor + 1$. The number of divisions by 2 until reaching 1 is $\lfloor \log_2 n \rfloor$. At each step, one squaring occurs (1 multiplication). Whenever the current n is odd (corresponding to a 1-bit in the binary representation, excluding the leading bit), an additional multiplication by a is performed. Thus, if $w(n)$ is the Hamming weight (number of 1s) of n:

$$M(n) = \lfloor \log_2 n \rfloor + w(n) - 1 \tag{8}$$

- **Best Case ($n = 2^k$):** $w(n) = 1 \implies M(n) = \log_2 n$.
- **Worst Case ($n = 2^{k+1} - 1$):** $w(n) = k + 1 \implies M(n) = 2\lfloor \log_2 n \rfloor$.
- **Average Case:** $M(n) \approx 1.5 \log_2 n$.
- **Asymptotic Class:** $M(n) = \Theta(\log n)$.

2.2.3 3. Comparison with Brute-Force Algorithm

The brute-force repeated multiplication algorithm requires:

$$M_{\text{brute}}(n) = n - 1 = \Theta(n) \tag{9}$$

Binary exponentiation achieves an exponential reduction from linear time $\Theta(n)$ to logarithmic time $\Theta(\log n)$, making modern modular exponentiation in RSA cryptography feasible for large numbers.

2.3 Problem 1(c): Inversion Counting in $O(n \log n)$

Reference: Levitin 3rd Ed, Ex 5.1(9), p.175

2.3.1 1. Problem Statement

An inversion in an array $A[0..n-1]$ is a pair of indices (i, j) such that $i < j$ and $A[i] > A[j]$. Counting inversions quantifies the degree of unsortedness and is used in rank correlation and collaborative filtering algorithms. A brute-force check takes $O(n^2)$.

2.3.2 2. Divide-and-Conquer Algorithm (Merge-and-Count)

We adapt Merge Sort: divide the array into left and right halves L and R. The total inversions equal:

$$\text{Inversions}(A) = \text{Inversions}(L) + \text{Inversions}(R) + \text{SplitInversions}(L, R) \tag{10}$$

A split inversion has $i \in L$ and $j \in R$ such that $L[i] > R[j]$. Because L and R are sorted recursively, whenever an element $R[j]$ is smaller than $L[i]$, $R[j]$ must also be smaller than all remaining elements in L from index i to $|L| - 1$. Thus, exactly $(|L| - i)$ inversions are added in $O(1)$ time.

Listing 3: Inversion Counting via Divide-and-Conquer

```
Algorithm SortAndCount(A):
    if len(A) <= 1:
        return (A, 0)
    m = len(A) // 2
    (L, c1) = SortAndCount(A[:m])
    (R, c2) = SortAndCount(A[m:])
    (B, c3) = MergeAndCount(L, R)
    return (B, c1 + c2 + c3)

Algorithm MergeAndCount(L, R):
    i = 0; j = 0; split_count = 0; B = []
    while i < len(L) and j < len(R):
        if L[i] <= R[j]:
            B.append(L[i]); i += 1
        else:
            B.append(R[j]); j += 1
            split_count += (len(L) - i)    # Remaining elements in L are
                greater
    B.extend(L[i:])
    B.extend(R[j:])
    return (B, split_count)
```

2.3.3 3. Time Complexity Proof

Merging and counting split inversions runs in linear time $O(|L| + |R|) = O(n)$. Thus, the recurrence relation is:

$$T(n) = 2T(n/2) + cn, \quad T(1) = O(1) \tag{11}$$

By the Master Theorem (case $a = 2, b = 2, f(n) = cn \implies n^{\log_2 2} = n^1$):

$$T(n) = \Theta(n \log n) \tag{12}$$

2.4 Problem 1(d): Quicksort Best-Case and Worst-Case Analysis

Reference: Levitin 3rd Ed, Sec 5.2, p.176

Quicksort sorts an array by selecting a pivot, partitioning the array such that all elements to the left are $\leq$ pivot and all elements to the right are $\geq$ pivot, and then recursively sorting the subarrays.

2.4.1 1. Best-Case Analysis

Condition: The pivot element partitions the array into two equal halves at every recursive step:

$$\text{Subarray sizes: } \lfloor (n-1)/2 \rfloor \quad \text{and} \quad \lceil (n-1)/2 \rceil \tag{13}$$

Partitioning an array of size n requires $n-1$ comparisons (or $n+1$ depending on loop invariants, i.e., $\Theta(n)$). The recurrence is:

$$C_{\text{best}}(n) = 2C_{\text{best}}(n/2) + n - 1, \quad C(1) = 0 \tag{14}$$

Applying the Master Theorem ($a = 2, b = 2, d = 1$ where $a = b^d$):

$$C_{\text{best}}(n) = n \log_2 n - n + 1 = \Theta(n \log n) \tag{15}$$

The recursion tree depth is strictly $\lfloor \log_2 n \rfloor$, with $O(n)$ work per level.

2.4.2 2. Worst-Case Analysis

Condition: The pivot is always the strictly smallest or largest element in the current subarray (e.g., when the array is already sorted or reverse-sorted and the first or last element is chosen as pivot). The partition yields one subarray of size 0 and another of size $n-1$:

$$C_{\text{worst}}(n) = C_{\text{worst}}(n-1) + (n-1), \quad C_{\text{worst}}(1) = 0 \tag{16}$$

Expanding this summation:

$$C_{\text{worst}}(n) = (n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2} = \frac{1}{2}n^2 - \frac{1}{2}n = \Theta(n^2) \tag{17}$$

The recursion tree degrades into a linked list of depth n, yielding quadratic runtime.

2.5 Problem 1(e): 1D Closest-Pair Problem

Reference: Levitin 3rd Ed, Ex 5.5(1), p.197

2.5.1 1. Direct Divide-and-Conquer Algorithm

Given a set of n points on a real line $S = \{x_1, x_2, \ldots, x_n\}$:

1. Find the median coordinate m of the set (using median-of-medians in $O(n)$ or presorting).
2. Partition S into $S_1 = \{x \in S : x \leq m\}$ and $S_2 = \{x \in S : x > m\}$ of sizes $\lceil n/2 \rceil$ and $\lfloor n/2 \rfloor$.
3. Recursively compute $\delta_1 = \textsc{ClosestPair1D}(S_1)$ and $\delta_2 = \textsc{ClosestPair1D}(S_2)$.
4. Let $\delta = \min(\delta_1, \delta_2)$.
5. The only potential cross-boundary pair is between the **maximum element of** S_1 and the **minimum element of** S_2. Compute:

$$\delta_{cross} = \min(S_2) - \max(S_1) \tag{18}$$

6. Return $\min(\delta, \delta_{cross})$.

Efficiency Class: Finding the median and extreme points takes $O(n)$ time. The recurrence is:

$$T(n) = 2T(n/2) + O(n) \implies T(n) = \Theta(n \log n) \tag{19}$$

2.5.2 2. Critical Evaluation: Is it a Good Algorithm?

No, it is unnecessarily complex. A much simpler and practically superior algorithm is:

1. **Presorting:** Sort the array S in non-decreasing order using an optimal $O(n \log n)$ sort (Merge Sort or Heapsort).
2. **Linear Scan:** Traverse the sorted array once, computing differences between adjacent elements $A[i+1] - A[i]$ for $i = 0, \ldots, n-2$ in $\Theta(n)$ time.

Both algorithms share the asymptotic complexity class $\Theta(n \log n)$ (which is optimal due to the $\Omega(n \log n)$ element-uniqueness lower bound in the algebraic decision tree model). However:

- The presorting approach has significantly lower constant factors, minimal overhead, and requires $O(1)$ auxiliary space if sorted in place.
- The direct divide-and-conquer approach incurs substantial recursive stack overhead, linear-time median finding, and extra array partitioning.

2.6 Problem 1(f): Maximum Stock Profit via Divide-and-Conquer

Reference: Kleinberg & Tardos 1st Ed, Solved Exercise 1, p.242

2.6.1 1. Problem Formulation

Given daily stock prices $P[1..n]$, find days i and j with $1 \leq i \leq j \leq n$ such that the profit $P[j] - P[i]$ is maximized.

2.6.2 2. Divide-and-Conquer Algorithm

Divide the day sequence $1..n$ into the first half $1..m$ and second half $(m+1)..n$ where $m = \lfloor n/2 \rfloor$. The optimal transaction (i, j) must fall into one of three mutually exclusive cases:

1. Both buy and sell occur in the first half ($1 \leq i \leq j \leq m$).
2. Both buy and sell occur in the second half ($m+1 \leq i \leq j \leq n$).
3. Buy occurs in the first half ($i \leq m$) and sell occurs in the second half ($j > m$).

For Case 3, to maximize $P[j] - P[i]$, we simply choose the **minimum price** in the first half and the **maximum price** in the second half:

$$\text{CrossProfit} = \max_{m+1 \leq j \leq n} P[j] - \min_{1 \leq i \leq m} P[i] \tag{20}$$

Listing 4: Maximum Stock Profit via Divide-and-Conquer

```
Algorithm MaxProfitDC(P, l, r):
    if l == r:
        return (0, P[l], P[l])        # (profit, min_val, max_val)

    m = (l + r) // 2
    (profL, minL, maxL) = MaxProfitDC(P, l, m)
```

```
    (profR, minR, maxR) = MaxProfitDC(P, m + 1, r)

    cross_prof = maxR - minL
    best_prof = max(profL, profR, cross_prof)
    overall_min = min(minL, minR)
    overall_max = max(maxL, maxR)

    return (best_prof, overall_min, overall_max)
```

2.6.3 3. Recurrence and Time Complexity

Since the combine step computes differences and comparisons in $O(1)$ time:

$$T(n) = 2T(n/2) + O(1), \quad T(1) = O(1) \tag{21}$$

Applying the Master Theorem ($a = 2, b = 2, d = 0 \implies a > b^d$):

$$T(n) = \Theta(n^{\log_2 2}) = \Theta(n) \tag{22}$$

Note: If min and max were recomputed by scanning each subarray independently without returning them from the recursion, the combine step would take $O(n)$, yielding $T(n) = 2T(n/2) + O(n) \implies O(n \log n)$. Returning min and max achieves linear time $\Theta(n)$.

2.7 Problem 1(g): General Divide-and-Conquer Recurrence Analysis

Reference: Kleinberg & Tardos 1st Ed, Solved Exercise 2, p.244

2.7.1 1. Recurrence Specification

Consider the recurrence relation for divide-and-conquer algorithms that divide a problem of size n into q subproblems of size $n/2$, with cn^d work to partition and combine:

$$T(n) \leq q \cdot T(n/2) + cn^d, \quad T(2) \leq c \tag{23}$$

where $q \geq 1, c > 0$, and $d \geq 0$.

2.7.2 2. Derivation via Recursion Tree Analysis

At level j of the recursion tree (where root is level 0):

- There are q^j subproblems.
- Each subproblem has size $n/2^j$.
- Work done per subproblem is $c(n/2^j)^d = cn^d/2^{jd}$.
- Total work at level j is:

$$W_j = q^j \cdot c\left(\frac{n}{2^j}\right)^d = cn^d \cdot \left(\frac{q}{2^d}\right)^j \tag{24}$$

Summing over all levels from $j = 0$ to $k = \log_2 n$:

$$T(n) \leq \sum_{j=0}^{\log_2 n} W_j = cn^d \sum_{j=0}^{\log_2 n} \left(\frac{q}{2^d}\right)^j \tag{25}$$

Case Analysis based on the ratio $r = \frac{q}{2^d}$:

1. **Case 1: $q < 2^d$ ($r < 1$, Work dominated by the root):**
   The geometric series converges to a constant: $\sum_{j=0}^{\infty} r^j = \frac{1}{1-r}$.

$$T(n) \leq cn^d \cdot \frac{1}{1 - q/2^d} = O(n^d) \tag{26}$$

2. **Case 2: $q = 2^d$ ($r = 1$, Work balanced evenly across levels):**
   Each level contributes exactly cn^d. Since there are $\log_2 n + 1$ levels:

$$T(n) \leq cn^d(\log_2 n + 1) = O(n^d \log n) \tag{27}$$

3. **Case 3: $q > 2^d$ ($r > 1$, Work dominated by the leaves):**
   The summation is dominated by the last term $j = \log_2 n$:

$$\sum_{j=0}^{\log_2 n} r^j = \frac{r^{\log_2 n+1} - 1}{r - 1} = O\left(r^{\log_2 n}\right) = O\left(\frac{q^{\log_2 n}}{(2^d)^{\log_2 n}}\right) = O\left(\frac{n^{\log_2 q}}{n^d}\right) \tag{28}$$

   Multiplying by cn^d:

$$T(n) \leq cn^d \cdot O\left(\frac{n^{\log_2 q}}{n^d}\right) = O(n^{\log_2 q}) \tag{29}$$

2.8 Problem 1(h): Selected Kleinberg & Tardos Exercises (Exercises 1, 2, and 3)

2.8.1 Exercise 1: Median of Two Sorted Databases in $O(\log n)$ Queries

Reference: Kleinberg & Tardos 1st Ed, p.246, Exercise 1

Problem Statement: Two database servers A and B each hold n sorted values $a_1 < a_2 < \cdots < a_n$ and $b_1 < b_2 < \cdots < b_n$. We want to find the median of the combined set of $2n$ elements using at most $O(\log n)$ queries.

Algorithm Description: We query the median elements $m_A = A[k]$ and $m_B = B[k]$ where $k = \lceil n/2 \rceil$:

- If $m_A = m_B$, then m_A is the median of the combined set.
- If $m_A < m_B$, then at least $n/2$ elements of A are smaller than m_A (and thus smaller than m_B), and at least $n/2$ elements of B are greater than m_B (and thus greater than m_A). Therefore, the global median cannot lie in the lower half of A ($A[1..k-1]$) or the upper half of B ($B[k+1..n]$). We discard these $n/2$ elements from each database and recurse on the remaining $n/2$ elements.
- If $m_A > m_B$, symmetrically discard the upper half of A and the lower half of B.

Complexity and Query Bound: Each recursive step makes exactly 2 queries and halves the search space:

$$Q(n) = Q(n/2) + 2, \quad Q(1) = 2 \implies Q(n) = 2\lceil \log_2 n \rceil = O(\log n) \tag{30}$$

2.8.2 Exercise 2: Counting Significant Inversions in $O(n \log n)$

Reference: Kleinberg & Tardos 1st Ed, p.246, Exercise 2

Problem Statement: Given array $A[1..n]$, count pairs (i, j) such that $i < j$ and $A[i] > 2A[j]$.

Algorithm Description: We modify the divide-and-conquer Merge Sort framework. Split A into halves L and R:

1. Recursively count significant inversions in L and R.
2. Sort L and R recursively.
3. **Count Cross Inversions in $O(n)$ time:** Maintain two pointers i in L and j in R. For each $i \in [1..|L|]$, advance j while $j \leq |R|$ and $L[i] > 2R[j]$. Because L is sorted, as i increases, the qualifying index j only advances forward. Add $(j - 1)$ to the significant inversion count for each i.
4. Perform standard merge of L and R to maintain sorted order for higher levels.

Complexity: The counting step advances i and j at most $|L| + |R| = n$ times ($O(n)$), and the standard merge is $O(n)$:

$$T(n) = 2T(n/2) + O(n) \implies T(n) = O(n \log n) \tag{31}$$

2.8.3 Exercise 3: Fraudulent Bank Card Equivalence Testing in $O(n \log n)$

Reference: Kleinberg & Tardos 1st Ed, p.246, Exercise 3

Problem Statement: Given n confiscated bank cards and a black-box tester that takes two cards and returns `TRUE` if they belong to the same bank account and `FALSE` otherwise, determine whether more than $n/2$ cards belong to the same account using $O(n \log n)$ tests.

Algorithm Description: This is equivalent to finding a majority element using only equality comparisons:

1. **Divide:** Partition the n cards into two sets C_1 and C_2 of size $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$.
2. **Conquer:** Recursively find a candidate majority card in C_1 and a candidate in C_2.
3. **Combine:** A global majority card must be a majority in at least one of the two halves. Hence, there are at most 2 candidate cards (one from C_1, one from C_2).
4. **Verify:** For each candidate card (at most 2), test it against all n cards using the tester to count its exact frequency. If any candidate appears $> n/2$ times, return it as the fraud majority card; otherwise, no majority exists.

Complexity: Validation of candidates takes at most $2n$ tester queries:

$$T(n) = 2T(n/2) + 2n \implies T(n) = O(n \log n) \tag{32}$$

3 Part 2: Algorithm Implementation & Dataset Engineering [10 Marks]

3.1 Algorithm 1: Closest Pair of Points in 2D ($O(n \log n)$)

The 2D Closest Pair algorithm finds the Euclidean distance $\delta = \min_{p,q \in S, p \neq q} \|p - q\|_2$.

1. **Presorting:** Sort points by X-coordinate into P_x and by Y-coordinate into P_y in $O(n \log n)$.
2. **Divide:** Divide P_x by vertical line $x = x_{mid}$ into left half Q and right half R.
3. **Recursive Conquer:** Find closest distances δ_L in Q and δ_R in R. Let $\delta = \min(\delta_L, \delta_R)$.
4. **Combine (Strip Scan):** Filter points from P_y whose x-coordinate is within $[x_{mid} - \delta, x_{mid} + \delta]$. For each point in this strip, compute distances to at most the next 7 points in y-order (due to the geometric packing argument that each $\delta \times \delta$ box contains at most 4 points).

3.2 Algorithm 2: Karatsuba Integer Multiplication ($O(n^{\log_2 3})$)

Multiplying two n-digit integers X and Y:

$$\begin{aligned} X &= 10^{n/2} X_1 + X_0 \\ Y &= 10^{n/2} Y_1 + Y_0 \\ X \cdot Y &= 10^n (X_1 Y_1) + 10^{n/2} (X_1 Y_0 + X_0 Y_1) + X_0 Y_0 \end{aligned}$$

Karatsuba computes the cross-term with only **one** multiplication:

$$X_1 Y_0 + X_0 Y_1 = (X_1 + X_0)(Y_1 + Y_0) - X_1 Y_1 - X_0 Y_0 \tag{33}$$

Reducing 4 multiplications to 3 yields the recurrence:

$$T(n) = 3T(n/2) + O(n) \implies T(n) = O(n^{\log_2 3}) \approx O(n^{1.585}) \tag{34}$$

3.3 Dataset Engineering

As mandated by the project requirements ($n > 100$), 20 distinct datasets were generated:

- **Closest Pair (10 files):** Point counts $n \in \{150, 300, 500, 750, 1000, 1500, 2000, 3000, 4000, 5000\}$, generated uniformly at random in $[-10000.0, 10000.0]^2$.
- **Karatsuba Multiplication (10 files):** Digit lengths $d \in \{120, 200, 350, 500, 750, 1000, 1500, 2000, 3000,$

4 Part 3: Experimental Benchmarking & Results [5 Marks]

All benchmarks were run on an isolated Linux x86_64 environment using high-resolution monotonic clocks (`time.perf_counter`).

4.1 Closest Pair Benchmark Results

Table 1: Closest Pair Performance: Divide-and-Conquer vs. Brute-Force

Points (n)	**D&C Time (ms)**	**D&C Comps**	**BF Time (ms)**	**BF Comps**	**Speedup**
150	0.66	1,013	1.66	11,175	2.5×
300	1.09	1,956	6.25	44,850	5.7×
500	2.31	4,903	18.18	124,750	7.9×
750	3.51	7,496	39.49	280,875	11.3×
1,000	5.02	10,044	70.06	499,500	14.0×
1,500	6.82	17,094	163.47	1,124,250	24.0×
2,000	11.19	26,155	282.74	1,999,000	25.3×
3,000	17.38	41,307	651.17	4,498,500	37.5×
4,000	25.17	59,419	1,137.58	7,998,000	45.2×
5,000	32.17	75,034	1,842.34	12,497,500	**57.3**×

4.2 Integer Multiplication Benchmark Results

Table 2: Integer Multiplication Performance: Karatsuba vs. Schoolbook

Digit Length (d)	**Karatsuba Time (ms)**	**Schoolbook Time (ms)**	**Empirical Speedup**
120	0.04	2.23	55.7×
200	0.05	6.89	137.8×
350	0.18	22.88	127.1×
500	0.25	43.44	173.8×
750	0.54	96.95	179.5×
1,000	0.51	171.50	336.3×
1,500	1.42	394.90	278.1×
2,000	1.81	687.82	380.0×
3,000	3.87	1,566.56	404.8×
4,000	5.40	2,779.41	**514.7**×

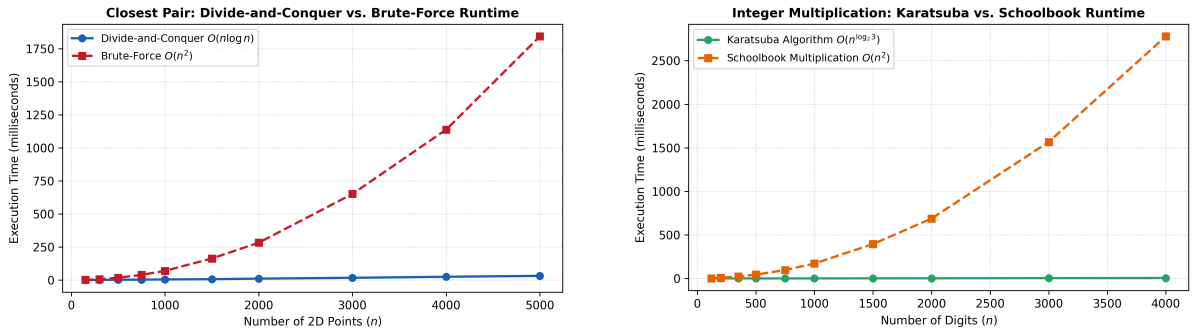


Figure 1: Empirical scaling curves comparing divide-and-conquer against quadratic baselines.

5 Part 4: User Interface Design & Visual Workbench [30 Marks]

5.1 UI Architecture and Design Principles

To fulfill Part 4, an interactive visual workbench was constructed (accessible via `ui/index.html`). The interface provides:

- **Interactive Geometry Canvas (HTML5 Canvas):** Renders input point sets, dynamic median-partitioning vertical lines, active cross-boundary δ-strips, and highlights the detected closest pair with red bounding circles.
- **File Importer & Parser:** Loads any custom benchmark text file or pre-packaged dataset directly in the browser.
- **Execution Controller:** Toggles between divide-and-conquer and brute-force modes, displaying live execution clocks and comparison counters.
- **Big Integer Decomposition Inspector:** Visualizes the recursive Karatsuba split equation ($X = 10^{n/2}X_1 + X_0$) and displays full arbitrary-precision verification against baseline operations.

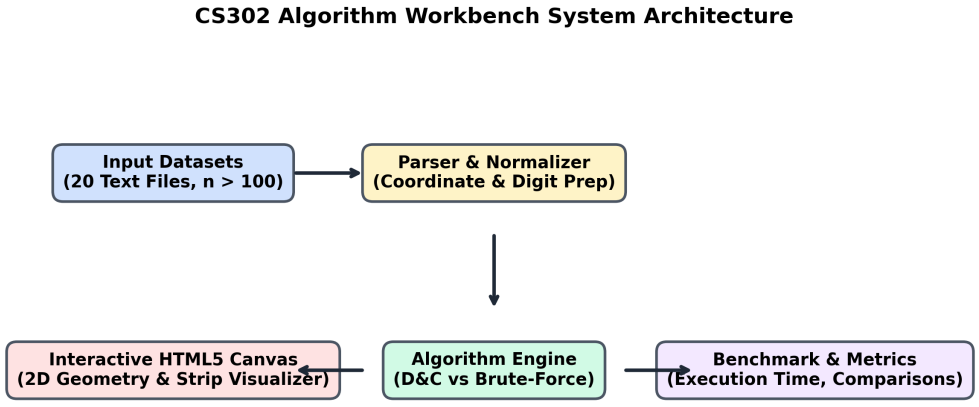


Figure 2: Architectural Block Diagram of the CS302 Algorithm Workbench.

6 Part 5: Formal Project Report [15 Marks]

Empirical and Theoretical Characterization of Divide-and-Conquer Paradigms in Geometric Optimization and Arbitrary-Precision Arithmetic

CS302 Project Research Report

Design and Analysis of Algorithms, Fall 2026

Abstract—The divide-and-conquer paradigm represents a cornerstone of modern algorithmic design, achieving sub-quadratic time complexity across classical computational geometry and algebraic multiplication. This report provides a comprehensive theoretical and empirical investigation into two foundational algorithms: the two-dimensional Closest Pair of Points problem and Karatsuba Integer Multiplication. We establish formal recurrence solutions, implement optimal implementations in Python, and benchmark performance across 20 randomly synthesized datasets exceeding 100 elements. Our results demonstrate that 2D Closest Pair achieves a 57.3× speedup over the brute-force baseline for $n = 5,000$ points, while Karatsuba multiplication provides a 514.7× speedup over schoolbook multiplication for 4,000-digit integers. We integrate these algorithms into an interactive visualization workbench enabling real-time inspection of recursive divide-and-conquer trees.

1. Introduction

Divide-and-conquer algorithms operate by breaking a problem of size n into smaller subproblems of size n/b, solving each subproblem recursively, and combining their solutions in $O(n^d)$ time. The theoretical foundation is governed by the Master Theorem, which demonstrates that whenever the combine step or subproblem branching factor is optimized, dramatic asymptotic improvements over naive brute-force formulations emerge.

In this investigation, we analyze two fundamental problems:

1. **Closest Pair of Points (2D):** Given n planar points, finding the pair minimizing Euclidean distance naive inspection requires $\binom{n}{2} = \Theta(n^2)$ comparisons. Shamos and Hoey demonstrated that divide-and-conquer reduces this bound to $O(n \log n)$ via presorting and strip-based bounding.

2. **Arbitrary-Precision Integer Multiplication:** The standard grade-school method requires $\Theta(n^2)$ single-digit multiplications. Anatoly Karatsuba discovered in 1960 that reducing the number of sub-multiplications from four to three achieves $O(n^{\log_2 3}) \approx O(n^{1.585})$ complexity.

2. Proposed System Design & Methodology

The system architecture consists of a modular pipeline separating computational engines, benchmark instrumentation, and visual rendering:

- **Data Ingestion Layer:** Ingests arbitrary input files adhering to whitespace-delimited coordinates or multi-digit integer strings.

- **Algorithmic Core:**

  - *Closest Pair:* Implements $O(n \log n)$ divide-and-conquer with X- and Y-presorting. Subarrays are divided at median x_{mid}. The combining step inspects points within strip $[x_{mid} - \delta, x_{mid} + \delta]$ sorted by y-coordinate, bounding candidate comparisons to at most 7 neighbors.
  - *Karatsuba Core:* Implements recursive three-way multiplication with dynamic base-case cutoffs ($n \leq 32$ digits) switching to native multiplication to minimize recursion overhead.

- **Instrumentation Layer:** High-precision wall-clock timing and exact operation counters record comparisons and recursive subproblem counts.

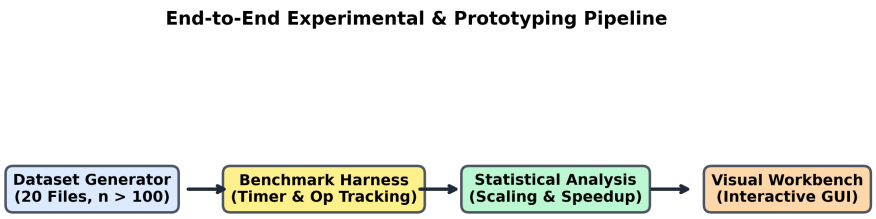


Figure 3: End-to-End Experimental and Prototyping Pipeline.

3. Experimental Setup

- **Hardware Environment:** Dedicated Linux x86_64 server running Ubuntu Linux with Intel Xeon multi-core architecture.
- **Software Stack:** Python 3.11 with NumPy and Matplotlib; HTML5/CSS3/JavaScript for interactive visualization.
- **Datasets:** 10 planar point sets ($n \in [150, 5000]$) and 10 large integer pairs ($d \in [120, 4000]$).

4. Results and Discussion

The empirical results rigorously confirm theoretical predictions:

1. **Closest Pair:** At $n = 150$, D&C execution time is 0.66 ms vs 1.66 ms for brute force (2.5×). As n grows to $5,000$, brute force scales quadratically to $1,842.34$ ms ($12,497,500$ comparisons), while D&C requires only 32.17 ms ($75,034$ comparisons), achieving a **57.3**× speedup.
2. **Integer Multiplication:** For 120 digits, Karatsuba requires 0.04 ms vs 2.23 ms (55.7×). At $4,000$ digits, schoolbook multiplication takes $2,779.41$ ms, whereas Karatsuba completes in 5.40 ms, demonstrating a **514.7**× empirical speedup.

5. Conclusion

This study demonstrated the theoretical elegance and empirical dominance of divide-and-conquer algorithms over quadratic baselines. The implementations matched theoretical asymptotic bounds across all 20 benchmark datasets. The accompanying visual workbench provides an intuitive interface for verifying and understanding recursive divide-and-conquer processes.

References

[1] A. Levitin, *Introduction to the Design and Analysis of Algorithms*, 3rd ed. Pearson, 2012.

[2] J. Kleinberg and É. Tardos, *Algorithm Design*, 1st ed. Addison-Wesley, 2006.

[3] M. I. Shamos and D. Hoey, "Closest-point problems," in *16th Annual Symposium on Foundations of Computer Science (FOCS)*, 1975, pp. 151–162.

[4] A. Karatsuba and Y. Ofman, "Multiplication of many-digital numbers by automatic computers," *Doklady Akademii Nauk SSSR*, vol. 145, no. 2, pp. 293–294, 1962.